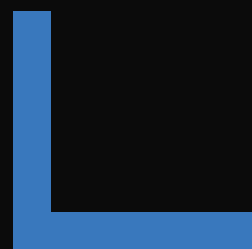




DuocUC[®] 

Autenticación Moderna
Con OAuth2 y OpenID Connect



Contenido

01

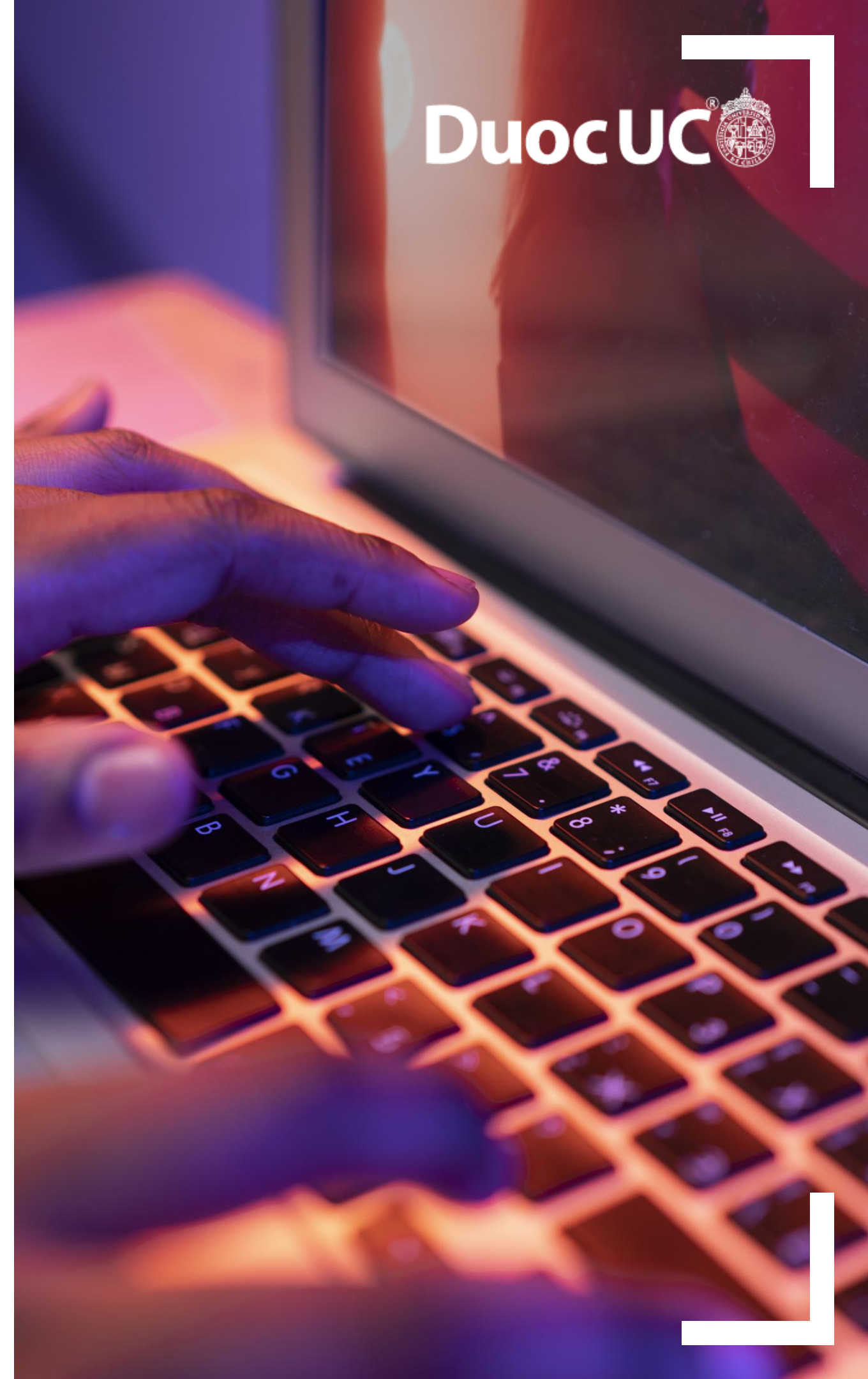
Autenticación
Moderna

02

OAuth y OIDC

03

Scope y
Endpoints



01

DuocUC 

Autenticación Moderna

Con OAuth2 y OpenID Connect

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex; flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
  color: white;  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```

¿Por qué necesitamos autenticación moderna?

Durante años la autenticación se basó en contraseñas propias de cada aplicación. Esto generó múltiples problemas: usuarios con demasiadas claves, almacenamiento inseguro de credenciales, ataques de fuerza bruta y fugas de datos. La tendencia actual es delegar identidad a proveedores externos confiables, usar tokens digitales seguros y evitar compartir contraseñas entre aplicaciones.



Evolución: De contraseñas a identidad

Antes: modelo basado en contraseñas

- Cada app gestionaba usuarios y contraseñas.
- Para que una app use datos de otra, debías compartir tu contraseña.
- Las apps almacenaban credenciales, aumentando riesgo de brechas.

Ahora: modelo basado en identidad

- La autenticación se delega a proveedores como Google, Microsoft o Auth0.
- Las aplicaciones ya no administran contraseñas, solo tokens.
- El usuario usa un inicio de sesión único (SSO) en múltiples plataformas.



02

OAuth y OIDC

SUBTÍTULO



Autenticación OIDC

- La autenticación responde: “¿**Quién eres realmente?**”. OpenID Connect permite validar la identidad del usuario mediante un **ID Token**, emitido por un proveedor confiable. Es equivalente al pasaporte digital del usuario. Contiene información como correo, nombre, identificador único y fecha de expiración. Las aplicaciones confían en este token sin tener que guardar contraseñas ni gestionar usuarios manualmente.

Autorización

OAuth2

La autorización responde: “¿**Qué puedes hacer?**”. OAuth2 permite que una app solicite acceso limitado a recursos protegidos, como datos en una API. Esto se realiza mediante un **Access Token**, que indica los permisos concedidos. Similar a una tarjeta de acceso, el token no identifica al usuario, pero sí determina qué operaciones están autorizadas.

Conceptos OAuth2 y OIDC

- Tokens: cadenas (normalmente JWT o string opaco) que representan derechos o permisos para acceder a un recurso

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImNsb3VkIG5hdG12ZSAxIiwicm9sZSI6InN0dWRlbnQiLCJpYXQiOiE3MzIzMTUyMDAsImV4cCI6MTczMjMxODgwMCwiaXNzIjoiaHR0cHM6Ly9hdXRoLmVqZW1wbG8uY2wiLCJhdWQiOiJodHRwczovL2FwaS5lamVtcGxvLmNsIn0.4JjE_t1LTiY9U05weiUEM5Zv5xeQfex5YLSgD7yYtmA
```

<> HEADER

Algoritmo

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

📄 PAYLOAD

Datos

```
{
  "sub": "1234567890",
  "name": "cloud native 1",
  "role": "student",
  "iat": 1732315200,
  "exp": 1732318800,
  "iss": "https://auth.ejemplo.cl",
  "aud": "https://api.ejemplo.cl"
}
```

* Datos decodificados del token real

🔒 SIGNATURE

Seguridad

```
HMACSHA256(
  base64Url(header) + "." +
  base64Url(payload),
  secret
)
```

Tokens que utiliza OAuth2 y OIDC

Refresh Token

- Permite obtener nuevos access tokens sin volver a pedir credenciales



Access Token

- Permite acceder a recursos protegidos (APIs) durante un tiempo corto



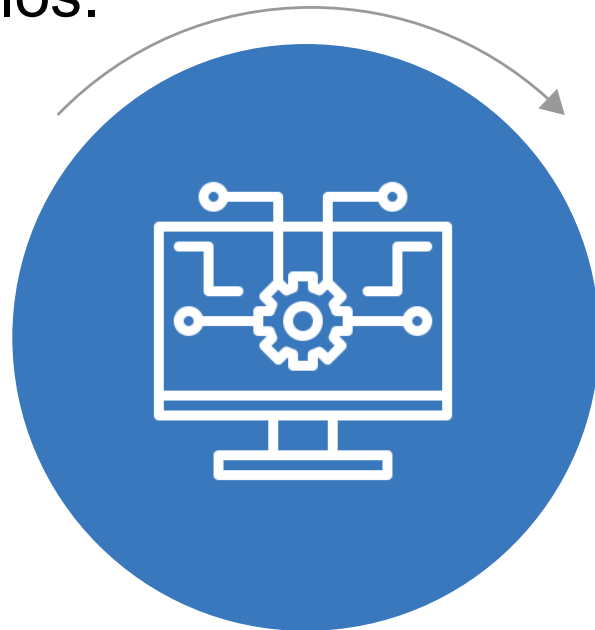
ID Token

- (Solo en OpenID Connect)
Identifica al usuario autenticado (JWT)

Los 4 Protagonistas de OAuth2

1.-Dueño del Recurso (Resource Owner)

El dueño de los datos.
Solo él puede otorgar
permiso para acceder a
ellos.



3. Servidor de Autorización

El sistema que verifica tu
identidad y emite los
tokens.



2. Cliente (Client)

La aplicación que **quiere
usar tus datos**.
Puede ser una app web,
móvil o un microservicio.



4.Servidor de Recursos

El lugar
donde viven tus
datos y tus APIs
protegidas.

03

Scopes y Endpoints

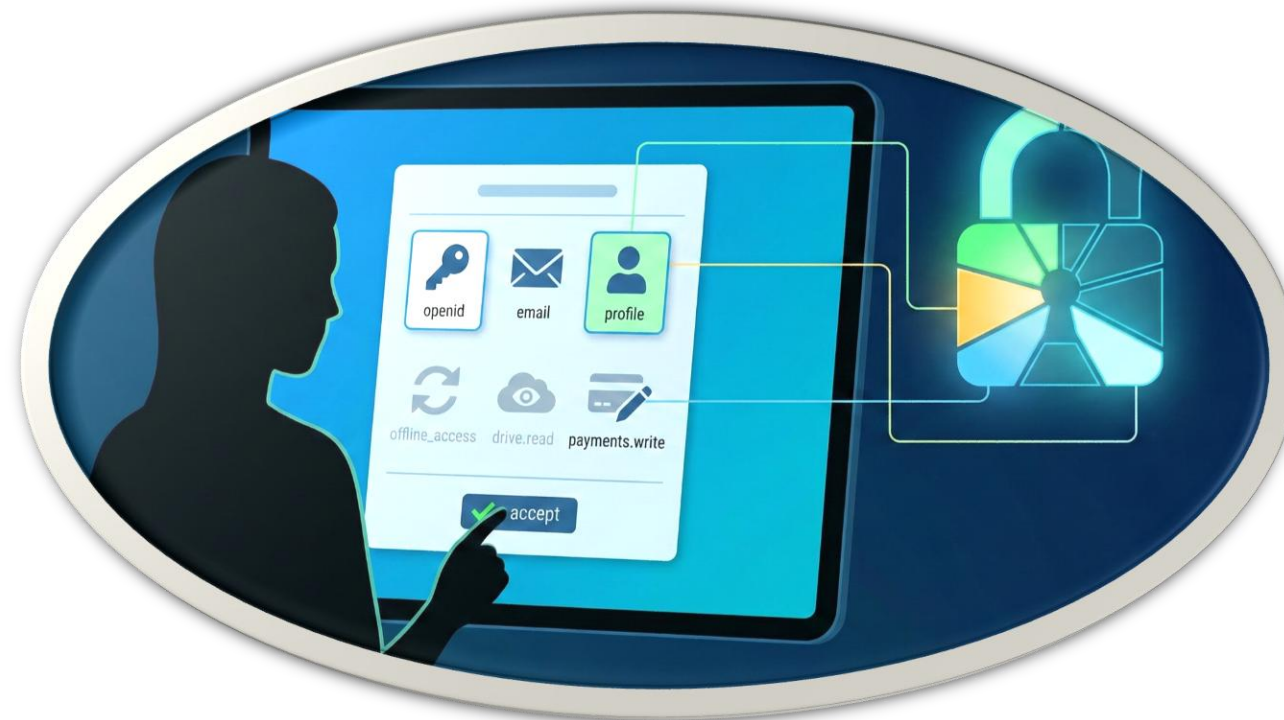
DuocUC 

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
  color: white;  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```

Scopes: los permisos en OAuth2

Los scopes definen el nivel de acceso que una aplicación solicita en OAuth2. Son un mecanismo para limitar el acceso de una aplicación a la cuenta del usuario, permitiendo granularidad

- **Openid:** obligatorio para OIDC.
- **email:** acceso al correo del usuario.
- **Profile:** datos básicos como nombre.
- **offline_access:** permite Refresh Tokens.
- Scopes personalizados como drive.read o payments.write. Los usuarios visualizan estos permisos en la pantalla de consentimiento.



Endpoints principales en OAuth2/OIDC

/authorize

Inicia el flujo de autenticación o autorización.

/token

Intercambia códigos por tokens.

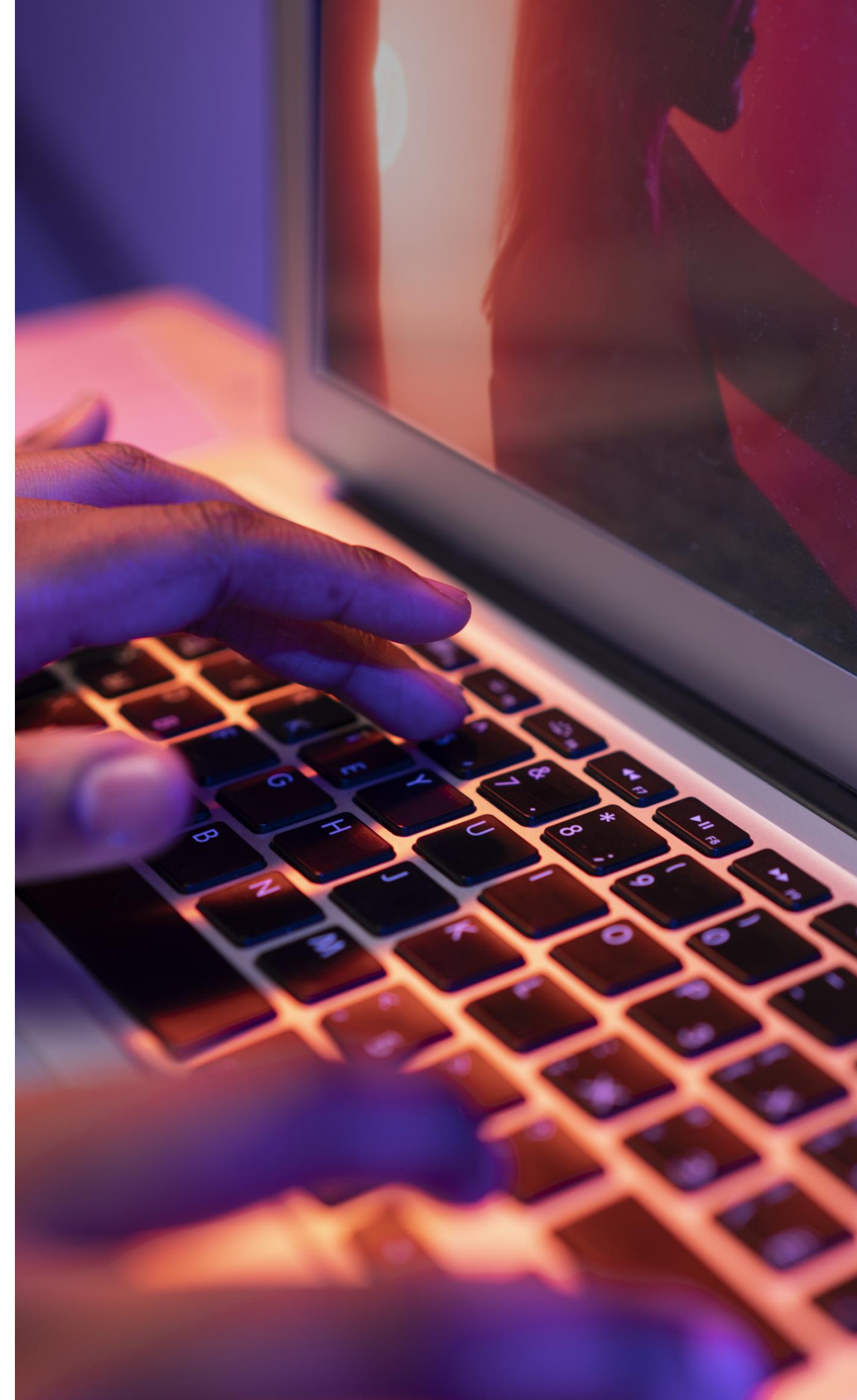
/userinfo

Devuelve la identidad del usuario (OIDC).

/jwks

Entrega claves públicas para verificar firmas de tokens JWT.

Estos endpoints permiten la comunicación estándar entre apps y el proveedor de identidad.



Flujos de OAuth2: Authorization Code con PKCE

El flujo Client Credentials se usa entre sistemas, no entre usuarios. Ideal para microservicios o automatizaciones.

Pasos simplificados:

- 1.El cliente envía su Client ID + Client Secret.
- 2.El Authorization Server verifica credenciales.
- 3.Devuelve un Access Token.
- 4.El servicio lo usa para acceder a APIs protegidas.

No hay intervención del usuario y no se emite ID Token.

Ejemplo completo: OIDC + OAuth2 con Google



Felicitaciones ¡Has finalizado el módulo!

Links de interés

OAuth 2.0 — RFC 6749

<https://datatracker.ietf.org/doc/html/rfc6749>

Bearer Token Usage — RFC 6750

<https://datatracker.ietf.org/doc/html/rfc6750>

JSON Web Key (JWKS) — RFC 7517

<https://datatracker.ietf.org/doc/html/rfc7517>

Scopes en OAuth2 — RFC 6749 (Section 3.3)

<https://datatracker.ietf.org/doc/html/rfc6749#section-3.3>

OAuth 2.0 Authorization Code + PKCE — RFC 7636

<https://datatracker.ietf.org/doc/html/rfc7636>

OAuth 2.0 Client Credentials Flow — RFC 6749 (Section 4.4)

<https://datatracker.ietf.org/doc/html/rfc6749#section-4.4>

Azure /authorize Endpoint (Auth Code Flow)

<https://learn.microsoft.com/azure/active-directory/develop/v2-oauth2-auth-code-flow#1-request-an-authorization-code>

Azure /token Endpoint (Auth Code Flow)

<https://learn.microsoft.com/azure/active-directory/develop/v2-oauth2-auth-code-flow#2-request-an-access-token>

JWT Authorizer en AWS API Gateway

<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html>

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación